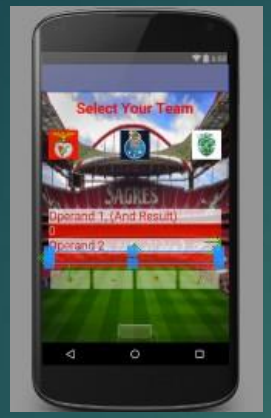
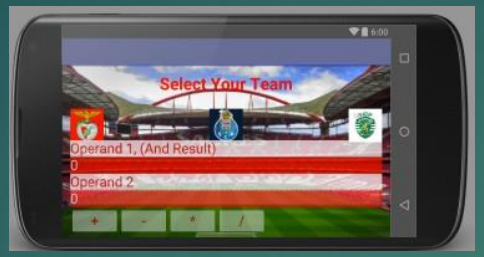
Android 03: Supporting Different Devices

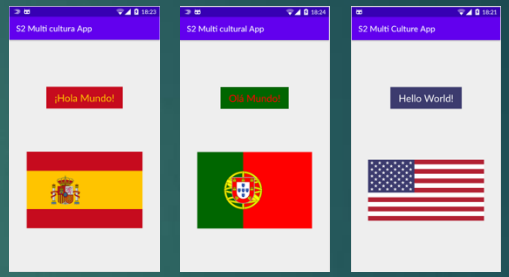
LEIM – DAM

Android app in different devices and for different cultures

- Android system is designed to be used on millions of smartphones, tablets and other devices for with a wide variety of screens and for user of any language and culture
  - Different **Languages and Cultures**, Different **Screens**, Different **Platform Versions**







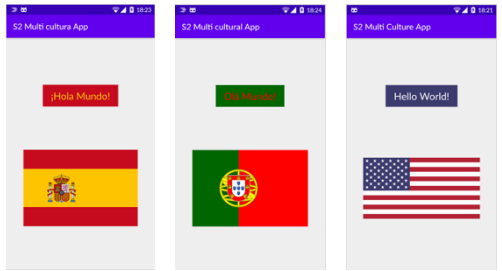
Alternative resources

Alternative resources

- Resources placed in res/ directory
  - Grouped in directories
    - Animator, anim, color *, drawable, mipmap, layout, menu, raw, values, xml, font

    * for color state list resources, like colours in different button states
  - Described in XML files
    - Inside values directory we may have: arrays.xml, colors.xml, dimens.xml, strings.xml , tyles.xml
- Alternative resources
  - Alternative resources provide a way to have appropriate resources in distinct cases
    - In distinct cultures (different language, images, sounds, …)
    - In distinct devices (different sizes, densities, …)

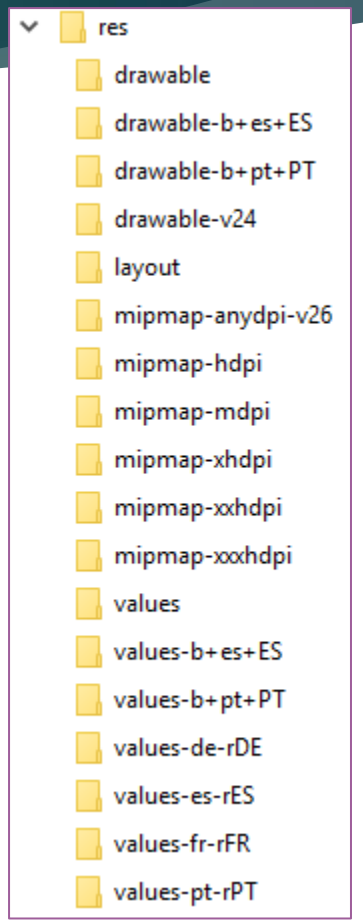
```
MyProject/
    src/
        MyActivity.java
    res/
        drawable/
            graphic.png
        layout/
            main.xml
            info.xml
        mipmap/
            icon.png
        values/
            strings.xml
```



https://developer.android.com/guide/topics/resources/providing-resources#AlternativeResources

Alternative resources

```
res/
    drawable/
        icon.png
        background.png
    drawable-hdpi/
        icon.png
        background.png
```



- Place for Alternative resources in directories
  - ***<resources_name>-<config_qualifier>***
  - *Resource_name: Animator, anim, color *, drawable, mipmap, layout, menu, raw, values, xml, font*
  - Type of Config_qualifier (for values see link below):
    - mobile country code, language and region, layout Direction, smallestWidth, Available width, Available height, screen size, Screen aspect, Round screen, Wide Color Gamut, High Dynamic Range (HDR), Screen orientation, UI mode, Night mode, Screen pixel density (dpi), Touchscreen type, Keyboard availability, Primary text input method, Navigation key availability, Primary non-touch navigation method, Platform Version (API level)

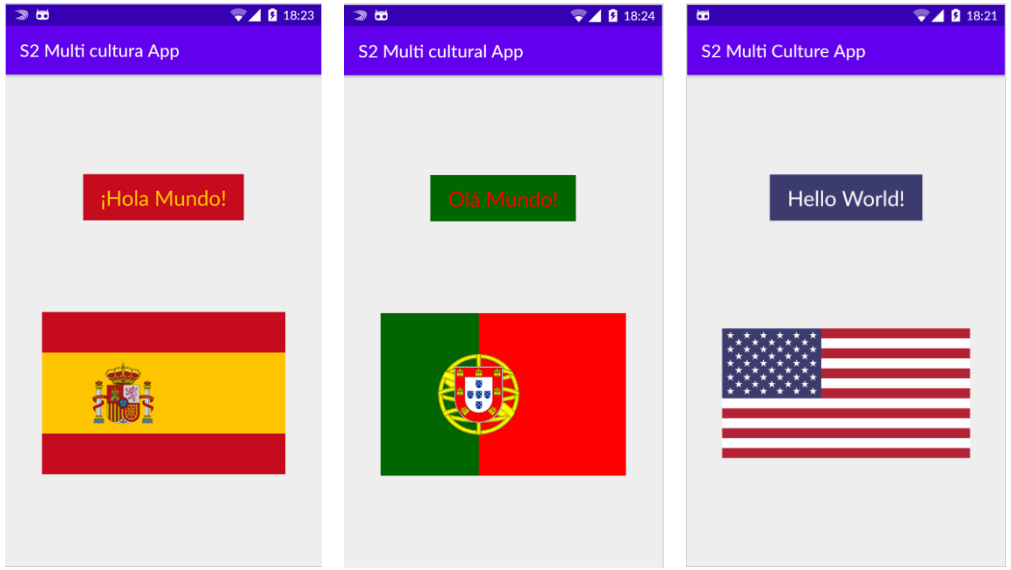
https://developer.android.com/guide/topics/resources/providing-resources#AlternativeResources

Different cultures

Different Languages

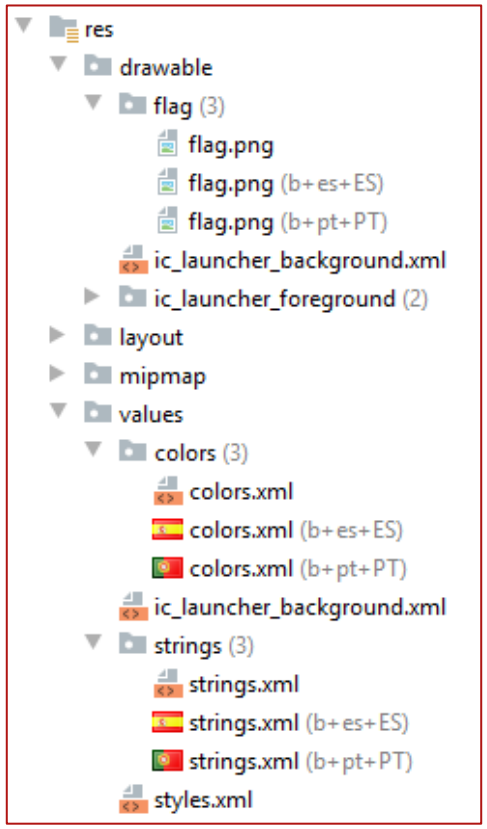
Idea: play country anthem

- Build a multi-culture app: an app that can be used by users with different languages and cultures
  - Different language, but also different images, colors, sounds, …
    - Flags extracted from wikipedia



Build a multi-culture app

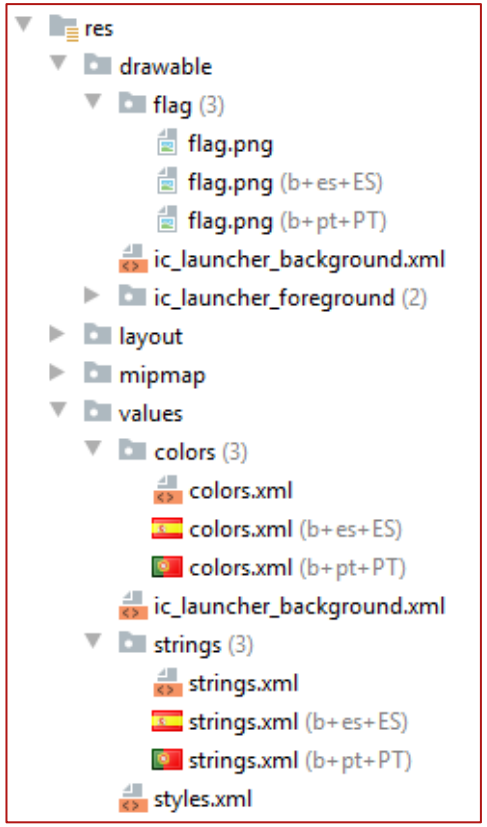
- In code use a reference to a resource (with an identifier)
  - Not use strings/colors/images/… directly in code
- Put resources and their identifiers in separate files
  - "res/values/strings" is the default file for en_US language
- Use other files for other languages (placed inside /res folder):
  - Files should have the **same name** (as original) **but** be placed **in a directory** that tells the system which culture to use:
    - `<resource type>-b+<language code>[+<country code>] or`
    - `<resource type>-<language code>[-r<country code>]`
  - Image on right shows Android view (Project Tool window) for previous app (different images, colors and Strings)
    - but files are in diferent directories – use Project view to see all files (and create folders)
  - The android system gets the resources depending on language activated in the device



Alternative resources - languages

- Alternative way to name alternative resource directories
  1. `<resource type>-b+<language code>[+<country code>] or`
  2. `<resource type>-<language code>[-r<country code>]`

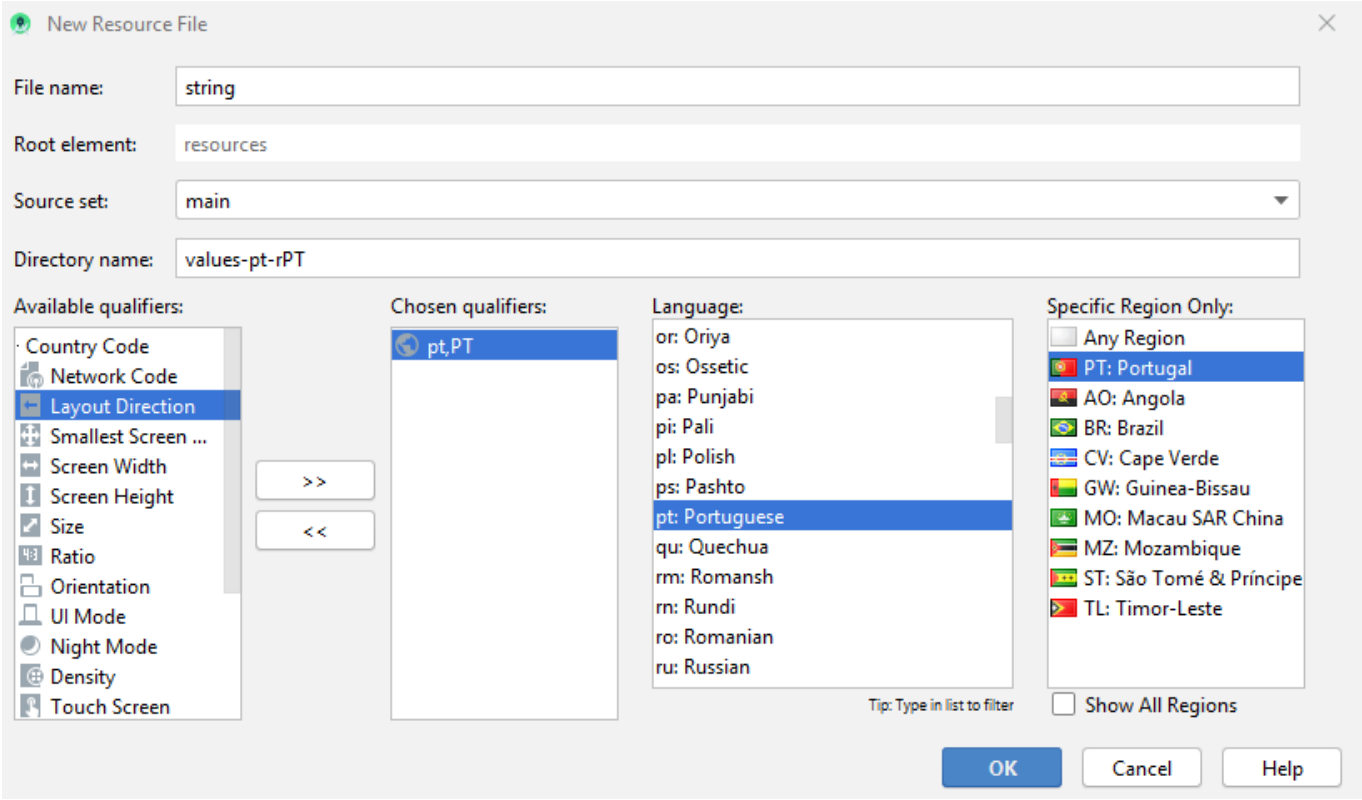
  - We can use both ways – the Android Studio uses the second way
    - The second way is the original Android way
    - The first way were introduced in Android 7, and uses a Best Practice from the IEFT (see link below)



- https://tools.ietf.org/html/bcp47 (BCP – Best current pratice) 47 Tags for Identifying Languages
- In Android we can also use: `<resource type>-<language code>[-r<country code>]`

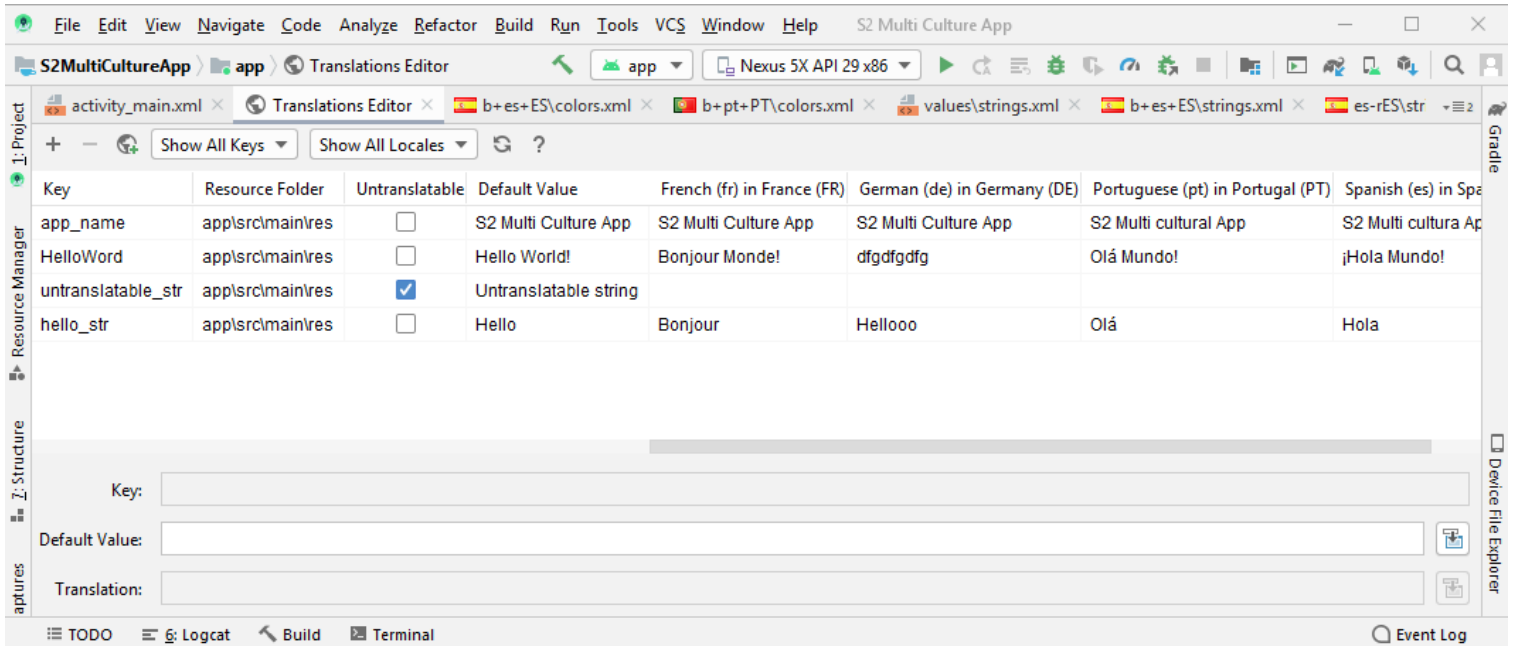
Create a Language resource file

- **Manually**: create the directory; create the Strings.xml file
- **Wizard**: over res/ select with the mouse: New -> Android resource File. On New Resource File window select the Language item and Language Settings, as in the figure



Create and manage Language resource files

- Use the Translations Editor:
  - over string with the mouse select "Open Translations Editor"
  - Here you can add/delete a Language file and a string item
  - This editor uses the Android nomenclature, ex: values-pt-rPT



strings.xml and colors.xml files for each Language

- English: res/values/strings.xml

```
<resources>
    <string name="app_name">S2 Multi Culture App</string>
    <string name="HelloWord">Hello World!</string>
</resources>
```

- Português: res/values-b+pt+PT/strings.xml

```
<resources>
    <string name="app_name">S2 Multi cultural App</string>
    <string name="HelloWord">Olá Mundo!</string>
</resources>
```

- Spanish: res/values-b+es+ES/strings.xml

```
<resources>
    <string name="app_name">S2 Multi cultura App</string>
    <string name="HelloWord">¡Hola Mundo!</string>
</resources>
```

colors.xml

```
<resources>
    <color name="colorPrimary">#6200EE</color>
    <color name="colorPrimaryDark">#3700B3</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="ColorCountry">#3C3B6E</color>
    <color name="ColorCountrySecond">#FDFDFD</color>
</resources>
```

```
<resources>
    <color name="colorPrimary">#6200EE</color>
    <color name="colorPrimaryDark">#3700B3</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="ColorCountry">#006600</color>
    <color name="ColorCountrySecond">#FF0000</color>
</resources>
```

```
<resources>
    <color name="colorPrimary">#6200EE</color>
    <color name="colorPrimaryDark">#3700B3</color>
    <color name="colorAccent">#03DAC5</color>
    <color name="ColorCountry">#C60B1E</color>
    <color name="ColorCountrySecond">#FFC400</color>
</resources>
```

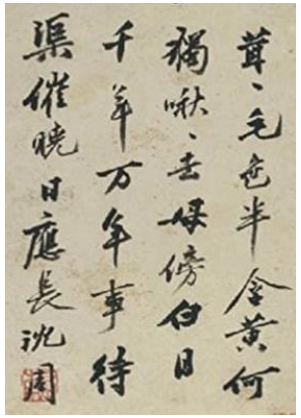
```
<TextView android:text="@string/HelloWord" android:textColor="@color/ColorCountrySecond"
          android:background="@color/ColorCountry" . . . />
<ImageView app:srcCompat="@drawable/flag" . . . />
```

Use in layout resource

Support for multiple languages and cultures

- Languages may use:
  - in right-to-left (RTL) text for Arabic or Hebraic: layout-ldrtl
    - instead of left-to-right (LTR) text - default
    - Layout Direction of: ldrtl or ldltr
  - in vertical columns which were read from top to bottom and right-to-left like in traditionally Chinese writings

والدهان والمخـرج والتـــاجـــر
والضـابط والحي والميت، ويومئ
الينـــا فـو از حـــديد عوسم ٤٩اذ
يهـــيئ حـــسن فكرت عـــشـــر
مسرحيات، ثم يومئ اذ تراود

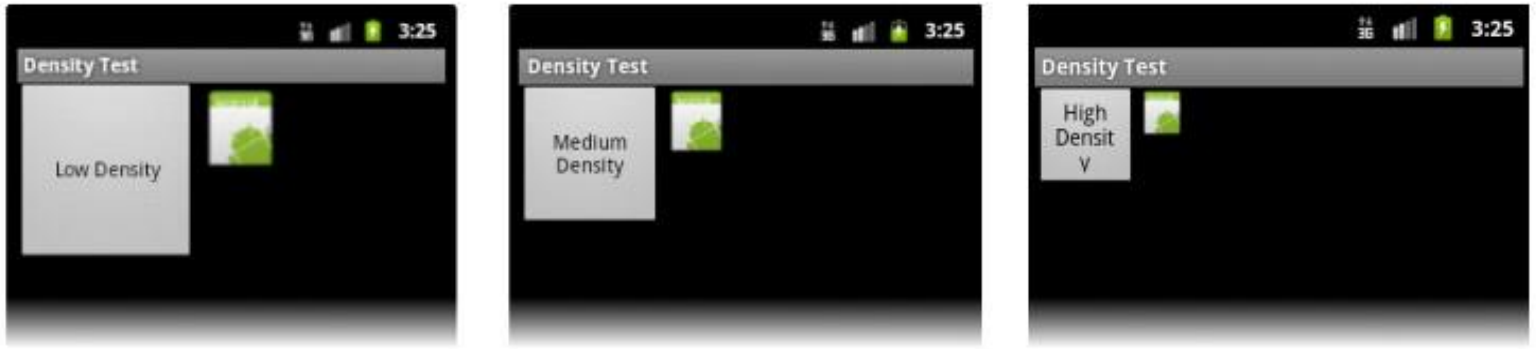


Different devices

Different Devices

- **Problems:**
  - Distorted images
  - Buttons, icons, and text too small or too large
  - Layouts adapted to small screens but not for large ones

- **How to develop applications to handle multiple devices?**



https://developer.android.com/guide/practices/screens_support

How to Support Multiple Screens?

- Provide **different layouts** for different screen sizes
  - Ex: layout-xlarge/
- Provide **different images** for different screen densities
  - Ex: drawable-hdpi/

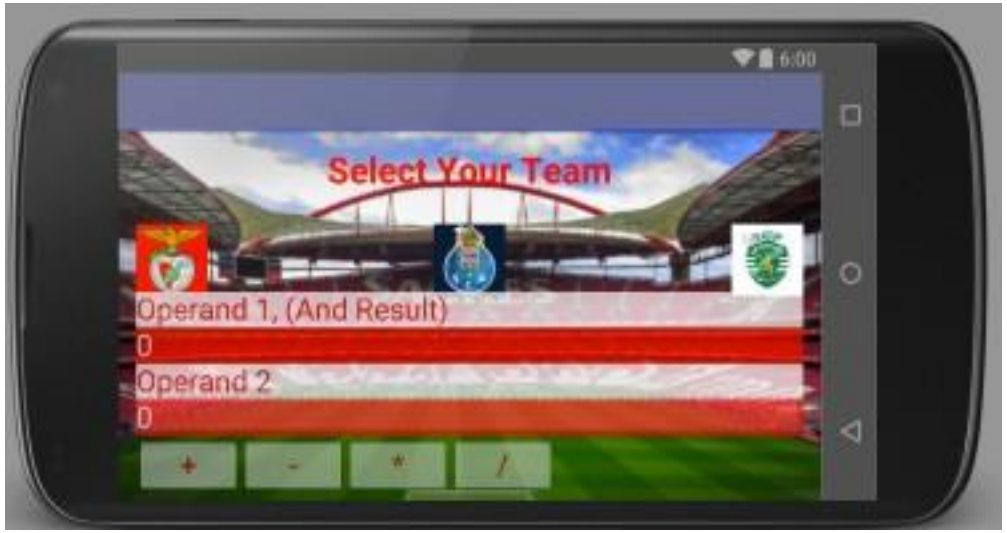
Features

- Orientation
- Display dimensions
- Display density
- Resolution
- Density-independent pixel (dp)
- Scale-independent pixel (sp)

Orientation

- Screen Orientation

Portrait

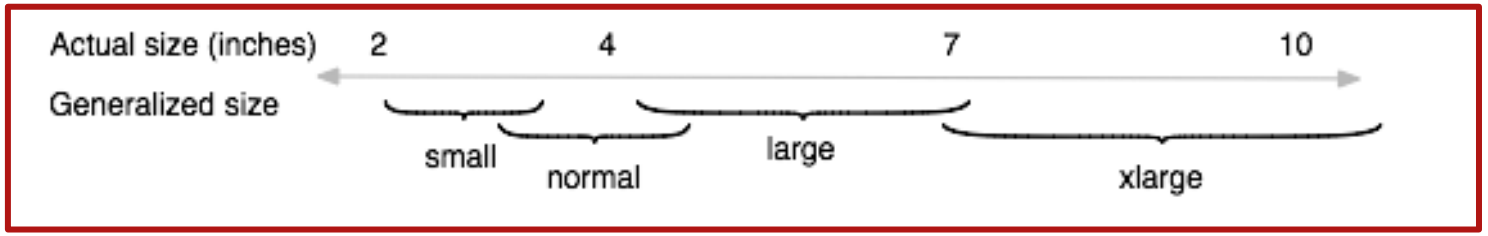


Landscape

Display size and group

- Screen physical size
  - Diagonal measurement in inches

- Devices grouped in 4 generalized sizes
  - **small**
  - **normal**
  - **large**
  - **extra-large**



Display Density

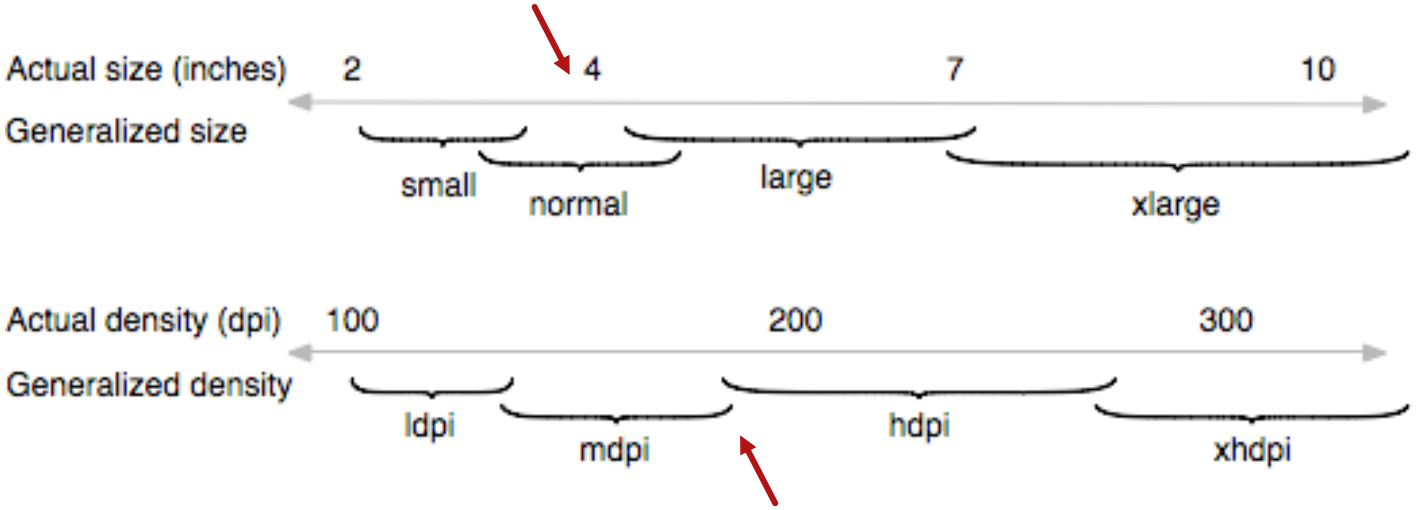
- Number of pixels per physical area of the screen
  - Unit of measurement - dpi (dots per inch)
  - Devices grouped into 6 generalized densities
    - ldpi: devices from 0 to 120 dpi
    - mdpi: devices from 121 to 160 dpi
    - …
  - Get screen size and density value, and size in dps:

```
DisplayMetrics dm = getResources().getDisplayMetrics();
int width = dm.widthPixels;
int height = dm.heightPixels;
int densityDpi = dm.densityDpi;
int width_dp = (width / densityDpi) * 160;
int height_dp = (height / densityDpi) * 160;
```

Group	Name	Density
ldpi	low density	120 dpi
mdpi	medium density	160 dpi
high	high density	240 dpi
xhdpi	extra high density	320 dpi
xxhdpi	extra extra high density	480 dpi
xxxhdpi	extra extra extra high density	640 dpi

Dimension and Density

- Base configuration (normal size and medium density)
  - First android device, T-Mobile G1
  - HVGA screen (3.2 inches, 320x480 pixels, 180 dpi)



Representative Screens

- The bold settings are available through the emulator

  * - set 120 dpi density

  ** - set 160 dpi density

	Low density (120), *ldpi*	Medium density (160), *mdpi*	High density (240), *hdpi*	Extra-high-density (320), *xhdpi*
Small screen	**QVGA (240x320)**		480x640	
Normal screen	**WQVGA400 (240x400)** **WQVGA432 (240x432)**	**HVGA (320x480)**	**WVGA800 (480x800)** **WVGA854 (480x854)** 600x1024	640x960
Large screen	**WVGA800** (480x800)** **WVGA854** (480x854)**	**WVGA800* (480x800)** **WVGA854* (480x854)** 600x1024		
Extra-Large screen	1024x600	**WXGA (1280x800)**† 1024x768 1280x768	1536x1152 1920x1152 1920x1200	2048x1536 2560x1536 2560x1600

Resolution (Dimension in pixels)

- Total number of pixels on the screen
  - To support multiple screens, applications should not handle pixel dimensions
- Problem
  - The same number of pixels on screens of different sizes produce different densities

Resolution and Screen Sizes: Solution

- Dimension values of layout elements in "**dp**" or with "**wrap_content**"
- **Alternative images** must be created for different densities
- Applications should use screen size and density as defined in **generalized groups**

Density-Independent Pixel

- Density-Independent Pixel (**dp**)
  - Virtual pixel units that must be used to define layouts (positions and dimensions) independently of the density
  - 1 dp = 1 px (pixel on the 160 dpi screen)
  - px = dp * (dpi / 160)

Scale-Independent Pixel

- Scale-Independent Pixel (**sp**)
  - dp = px * ratio
    - ratio = 160 / dpi
  - sp = dp * scale
  - sp = px * ratio * scale
  - **Scale** refers to text font size (**size** assigned by the user)
  - Must be used to set the **text size**

How to Support Multiple Screens?

1. Explicitly declare in Manifest the sizes of the **supported screens**
   - `<supports-screens>`
2. Provide **different layouts** for different screen sizes
   - `layout-xlarge/`
3. Provide **different images** for different screen densities
   - `drawable-hdpi/`

Multiple screen: 1 - Manifest Declaration

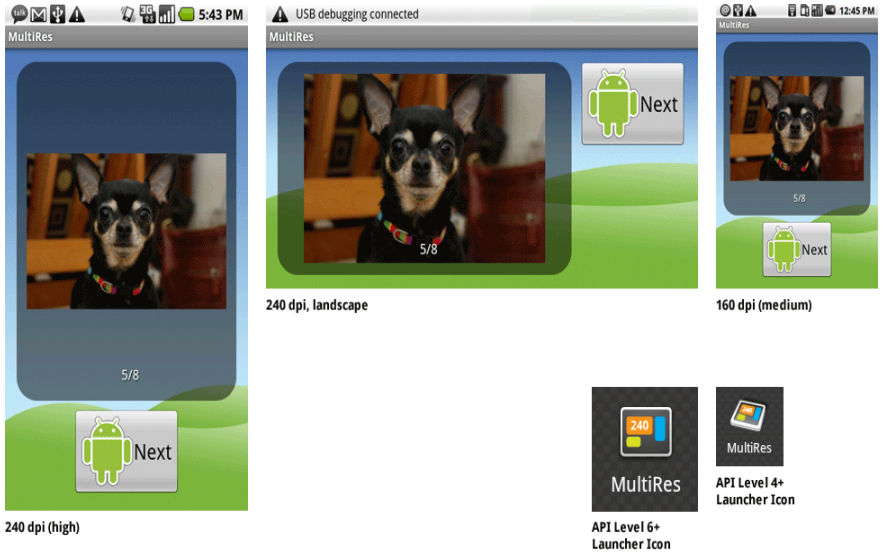
- Supported screen sizes declared in Manifest file
  - When the screen size is larger than the size and smaller than the smallest size allowed the Android system uses the compatible screen mode (scale the baseline).
  - Bold mean default values. The last three elements were introduced in version 13 (Android 3.2) and should be used instead of the first four ones

```
<supports-screens
    android:smallScreens=["true" | "false"]
    android:normalScreens=["true" | "false"]
    android:largeScreens=["true" | "false"]
    android:xlargeScreens=["true" | "false"]
    android:anyDensity=["true" | "false"]
    android:requiresSmallestWidthDp="integer"
    android:compatibleWidthLimitDp="integer"
    android:largestWidthLimitDp="integer"/>
```

Multiple screens: 2 - Diferent Layouts

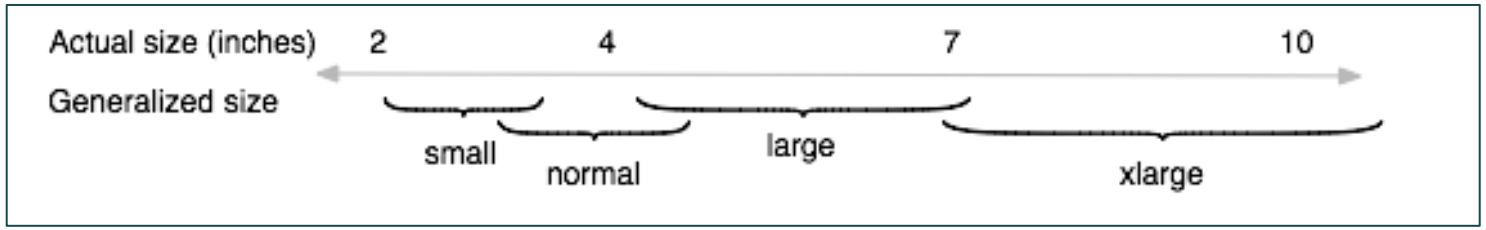
- Create a layout (xml file) for each screen size and landscape
  - content_main.xml
- Each layout should be placed on a different alternative directory
  - res/layout-<screen_size>

```
res/
  layout/                # default (portrait)
    content_main.xml
  layout-land/           # landscape
    content_main.xml
  layout-large/          # large (portrait)
    content_main.xml
  layout-large-land/     # large landscape
    content_main.xml
```



Multiple screens:
2 - Diferent Layouts, Display dimensions

- Screen physical size
  - Diagonal measurement in inches
- Android grouped in 4 generalized sizes
  - small
  - normal
  - large
  - extra-large



Multiple screens:
2 - Different Layouts: Problem

- Smartphone 5” (16:9) and Tablet 7” (4:3)
  - Same generalized group (large group) but on Tablet have more space
  - Android 3.2 to Android 7.0
  - Method based on the width and/or height of the layout defined in dp

```
res/layout/content_main.xml            # Smartphone
res/layout-sw600dp/content_main.xml    # Tablet
```

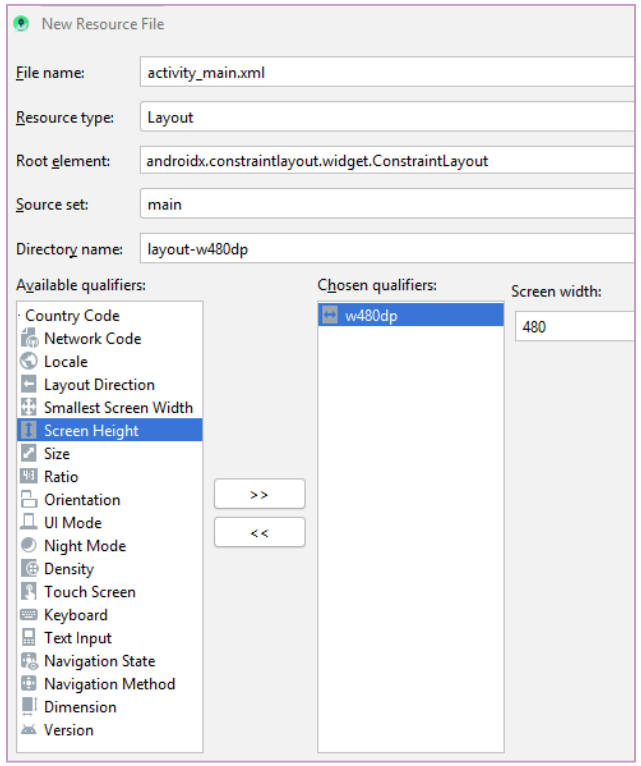
Multiple screens: 2 - Diferent Layouts

- Different Layouts: Android 3.2 to Android 6.0
- Examples

```
res/
  layout/content_activity.xml          # smartphone (smaller than 600dp available width)
  layout-sw600dp/content_activity.xml  # tablets 7” (600dp min. to the smallest dimension)
  layout-sw720dp/main_activity.xml     # tablets 10” (720dp min. to the smallest dimension)
  layout-w600dp/main_activity.xml      # Multi-pane (720dp min. for width)
  layout-h720dp/main_activity.xml      # 720dp minimum for height)
```

Multiple screens: 2 - Diferent Layouts, Typical Screen

- Add an alternative layout in Android Studio
  - Over Res/ directory with mouse select New – Android Resource File
    - In Resource type select the resource type:
      - Layout, in this case
    - Select the desired qualifiers and values:
      - Screen Width with 480dp in figure case



Screen Width	Settings
320 dp	<5” phone (240x320 ldpi, 320x480 mdpi, 480x800 hdpi, …)
480 dp	5” phone (480x800 mdpi)
600 dp	7” tablet (600x1024 mdpi)
720 dp	10” tablet (720x1280 mdpi, 800x1280 mdpi,…)

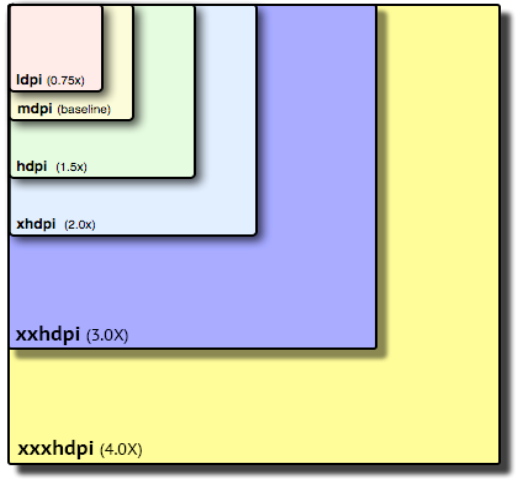
https://developer.android.com/training/multiscreen/screensizes

Multiple screens:
2 - Diferent Layouts, summary

- When should we create alternative layouts?
  - The application does not fit well with small screens
  - Not optimized to take advantage of additional space on large screens
  - Not optimized for both orientation (landscape and portrait)

Multiple screens: 3 – Alternative images

- Create alternative images for the 6 generalized densities, maintaining the relation between the scales. Example with an image with (pixels):
  - 36x36 - ldpi
  - 48x48 - mdpi
  - 72x72 - hdpi
  - 96x96 - xdpi
  - 180x180 - xxdpi
  - 192x192 - xxxdpi
- For all (app) images
  - Background images, menu icons, status bar icons, application icon



Multiple screens: 3 – Alternative images, organization

- Image resources in directories -xhdpi, -hdpi, …
  - Examples with an Image and Application icon

Image

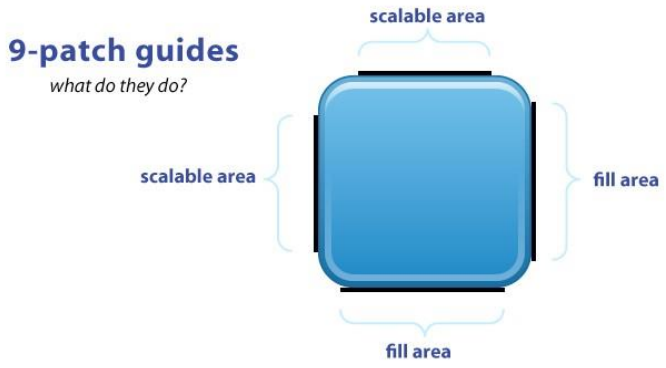
```
res/drawable-xhdpi/
    awesomeimage.png
res/drawable-hdpi/
    awesomeimage.png
res/drawable-mdpi/
    awesomeimage.png
res/drawable-ldpi/
    awesomeimage.png
```

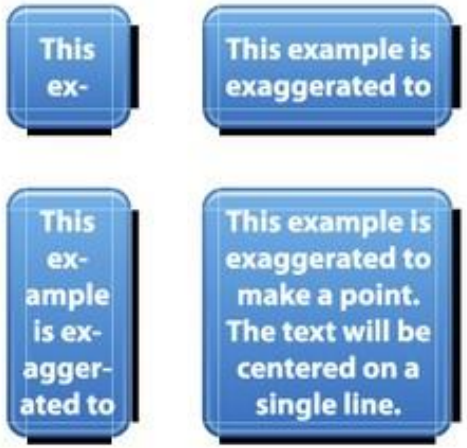
Application icon

```
res/mipmap-ldpi/
    finished_launcher_asset.png
res/mipmap-mdpi/
    finished_launcher_asset.png
res/mipmap-hdpi/
    finished_launcher_asset.png
. . .
```

Multiple screens:
3 – Alternative images, 9-Patch Images

- 9-Patch Images (ficheiros .9.png)
  - Allows you to scale without distortion
  - They have defined regions that can be stretched/shrunk and those that remain
  - Created using the draw9patch tool (Android SDK)
  - They should be used in the background images of the buttons





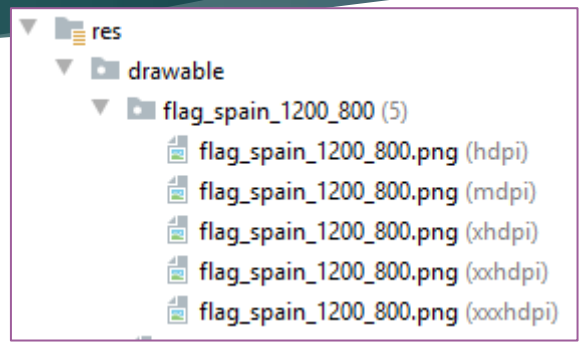
Multiple screens:
3 – Alternative images, Best Practices

- Use **wrap_content**, **match_parent**, or **dp** units in layout element dimensions
  - Do not use pixel values in the application
- Use **ConstrainLayout**
  - Use (percentual) GuideLines; use dimensions set to 0dp to stretch
  - Do not use the AbsoluteLayout element
- Create **alternative images** for different screen densities

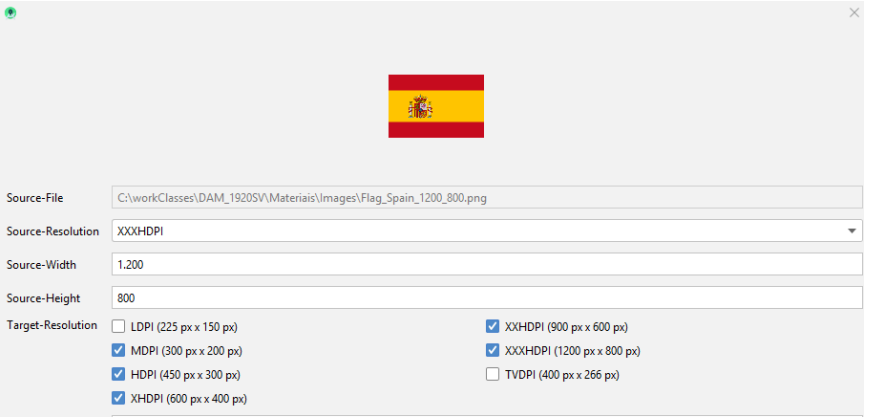
Multiple screens:
3 – Alternative images, Best Practices

Rule	Explanation
Compose / modern Android UI: mostly vectors + dp + adaptive layouts	Jetpack Compose layouts are defined in dp and scale automatically across densities, while vector drawables (.xml) ensure icons and simple graphics render sharply on any screen without needing multiple bitmap resources.
Launcher icons: provide proper density-ready assets/adaptive icons	Launcher icons should be provided as **adaptive icons** (foreground + background) in the mipmap-* folders so the system can render them correctly across launchers, shapes, and screen densities.
Large raster artwork: provide selected bitmap densities if the image looks soft when scaled	For photos or detailed illustrations, providing at least a **baseline (mdpi) and a higher density (e.g., xhdpi or xxhdpi)** version helps avoid blur while still keeping the APK size reasonable.
Don't overproduce assets unless your app is very visual, brand-sensitive, or image-heavy	Each additional density resource increases APK size and maintenance effort, so it is usually better to rely on scaling unless visual fidelity is critical for the user experience.

Android Studio automatic creation of alternative images



- Use **Andriod Drawable Importer** plugin
  - Video: https://www.youtube.com/watch?v=bbFLDR6X2D8
  - However the plugin in Android plugin repositor it is not working
    - https://plugins.jetbrains.com/plugin/7658-android-drawable-importer
    - But we can install this zip as a local file
      - File -> Settings -> Plugins:: in choose Install Plugin from disk… Browse for zip file, install it and restart Android Studio (wait for a long restart).
      - Direct link: https://gpskaihu.nz/ADI-hack/ADI-hack-plugin-AS36.zip
      - To use it: over **res** folder choose New -> Batch Drawable Import



2026 March – not checked

Seeing alternative images

- Na app with just one ImageView with the imported image
  - layout_width and height of 0dp (match constraints) and scaleType fitCenter
  - Images with an added identifier (l, m, h, x, xx, xxx)

Nexus 9

OnePlus One

Nexus 6

Bibliography

- Different Languages
- Multiple Screens
- Draw9patch
- Density-independent Pixels
- Alternative Resources
- Image Asset Studio
- Device Metrics